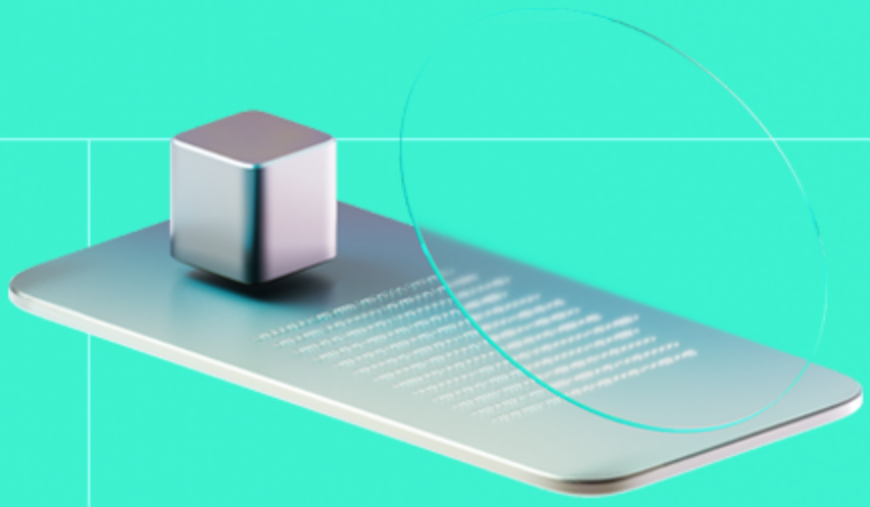
HACKEN

Smart Contract Code Review And Security Analysis Report



Customer: A Two Tech Limited

Date: 04/12/2025

We express our gratitude to the A Two Tech Limited team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

A Two Tech Limited is a decentralized, interactive platform transforming the nature of sports and entertainment by allowing fans to play an active, tokenized role in real-time event outcomes.

Document

Name	Smart Contract Code Review and Security Analysis Report for A Two Tech Limited
Audited By	, Khrystyna Tkachuk
Approved By	Ivan Bondar
Website	http://arenatwo.com/
Changelog	24/11/2025 - Preliminary Report
	04/12/2025 - Final Report
Platform	BSC
Language	Solidity
Tags	Vesting, Token Sales, Staking, Voting, Upgradable, ERC20, ERC721
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/arenatwo/smart-contracts/tree/hacken-2025-10-31
Initial Commit	3d8f05b
Remediation Commit	ca89ed1

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

13	11	1	1
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	5
Low	3

Vulnerability	Severity	Status
F-2025-14057 - Recover Function Can Steal User Funds	Medium	Fixed
F-2025-14062 - Excessive Admin Control Over Critical Staking Parameters	Medium	Mitigated
F-2025-14066 - Excessive Emergency Withdraw Can Steal User Funds	Medium	Fixed
F-2025-14071 - No On-Chain Vote Tallying	Medium	Accepted
F-2025-14074 - Admin Can Arbitrarily Decrease User's Vesting Amount Bypassing User Entitlement	Medium	Fixed
F-2025-14058 - Initialization and Address Validation Issues	Low	Fixed
F-2025-14065 - Use of Unsafe transferFrom Instead of safeTransferFrom	Low	Fixed
F-2025-14073 - Missing Reserve Check Allows Creation of Underfunded Vesting Schedules	Low	Fixed
F-2025-14067 - Floating Pragma	Info	Fixed
F-2025-14070 - Use of `public` Visibility Instead of `external` for External-Only Functions	Info	Fixed
F-2025-14072 - Unused Custom Error and Import	Info	Fixed
F-2025-14076 - Lack of Event Emitting for Key State Changes	Info	Fixed
F-2025-14077 - Redundant Token Address Comparison in recover() Function	Info	Fixed

Documentation quality

- Functional requirements are partially missed.
- Technical description is provided.
  - Run instructions are provided.
  - Technical specification is provided.
- Inline comments exist partially throughout the codebase.

Code quality

- The code leverages OpenZeppelin and Chainlink contracts and follows established patterns.
- The codebase is well-structured and clearly organized.
- The development environment is configured.

Test coverage

Code coverage of the project is **100%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is partially provided.
- Interactions by several users are not tested thoroughly.

Table of Contents

System Overview

A Two Tech Limited is an integrated, interactive ecosystem that brings fans into the decision-making process of live sports and entertainment events. Built with a modular architecture and driven by the $ATWO token, the platform allows fans to influence outcomes in real time, stake tokens to support teams, and share in the upside of the events they help shape.

The protocol includes the following contracts:

- **ATWO.sol** – ERC20 utility token for the protocol with burn capability and standard token operations. Implements a fixed-supply ERC20 with optional burning by holders. No minting or supply expansion is possible after deployment. It has the following attributes:
  - Name: ATWO
  - Symbol: ATWO
  - Decimals: 18
  - Total supply: 1,000,000.
- **ATWOMemberNFT.sol** – ERC721 membership NFT used to gate staking, league participation, and membership-restricted actions. Provides role-protected minting and baseURI management. Represents membership for a specific team within a league; all tokens share a unified base URI.
- **ATWOPresale.sol** – ATWO presale contract enforcing start/end windows, per-token pricing, and optional ATWO hard caps. Records contributions but does not distribute tokens; purchasers later claim via ATWODistributor. Supports approved payment tokens and configurable treasury forwarding.
- **ATWOPublicSale.sol** – public sale contract enabling immediate ATWO distribution upon purchase. Supports allowed payment tokens, price configuration, start-time gating, and hard-cap enforcement. Requires the contract to be pre-seeded with ATWO; collected funds remain in the contract until recovered by admin.
- **ATWOStaking.sol** – staking contract that requires membership NFT eligibility and league-specific staking windows. Allows staking ATWO per team within a league, blocks unstaking during active league windows, tracks per-team/user stake balances, and routes membership fees to team treasuries.
- **ATWOVoting.sol** – ATWO-based voting contract with per-question pricing. Admins manage question lifecycle and enable/disable options. Users pay ATWO to vote; fees are forwarded to a treasury. Vote counts themselves are tracked off-chain via emitted events.
- **ATWODistributor.sol** – distributor/claimer contract for presale participants. Holds ATWO and releases tokens after the TGE timestamp. Reads entitlement amounts directly from ATWOPresale and allows users to claim 100% of their purchased tokens once claims open.

Privileged roles

The protocol uses OpenZeppelin's `AccessControl` system with the following role hierarchy:

`DEFAULT_ADMIN_ROLE`: The root administrator role with the highest level of access. This role can grant and revoke all other roles in the protocol.

`ADMIN_ROLE`: Primary operational and configuration role for all upgradeable modules. Depending on the contract, `ADMIN_ROLE` is responsible for:

- **Protocol configuration**: updating parameters and administrative settings.
- **Upgrade control**: authorizing UUPS upgrades.
- **NFT management**: modifying base URIs, metadata, supply limits, rarity thresholds, and VRF configuration.
- **Minting control**: minting NFTs directly (including bypassing VRF for specific rarities where applicable).
- **Sale management**: setting sale prices, start times, supply, payment tokens, and treasury destinations.
- **Membership system**: configuring membership prices, team treasuries, and membership NFT addresses.
- **League/staking control**: defining league windows and controlling when staking/unstaking is allowed.
- **Voting administration**: opening/closing questions, enabling options, setting vote prices, and configuring vote-fee treasury.
- **Presale/public sale**: setting pricing, payment tokens, hard caps, seeding tokens, and recovering funds.
- **Distributor**: setting TGE time, seeding tokens for distribution, and recovering balances.
- **Vesting**: creating or modifying vesting schedules and executing emergency withdrawals.

`MINTER_ROLE`: Role intended for controlled, permissioned minting of NFTs. Responsibilities include:

- **Membership NFTs**: minting membership tokens required for staking participation.
- **OG Pass (Rarity)**: minting NFTs via the standard VRF process or batch-airdropping predefined rarities.
- **OG Pass (Simple)**: minting non-rarity NFTs (e.g., Champion passes).

`PAUSER_ROLE`: Emergency-response role with authority to halt protocol functionality. Holders of this role can:

- Call `pause()` on any pausable contract.
- Immediately stop all user-facing operations such as purchases, sales, staking, voting, vesting claims, and token distribution.
- Use this capability during security incidents or critical failures.

Administrative functions remain available while paused.

`UNPAUSER_ROLE`: Role that can unpause contracts after they have been paused, restoring normal operations after an emergency pause.

Potential Risks

- **Centralized Minting to a Single Address**: At initialization of `ATWO` contract, all tokens are minted to a single wallet. If this wallet is compromised, all tokens could be at risk.
- **Dynamic Array Iteration Gas Limit Risks**: The project iterates over large dynamic arrays, which leads to excessive gas costs, risking denial of service due to out-of-gas errors, directly impacting contract usability and reliability.
- **Owner's Unrestricted State Modification**: The absence of restrictions on state variable modifications by the owner leads to arbitrary changes, affecting contract integrity and user trust, especially during critical operations like minting phases.
- **Single Points of Failure and Control**: The project is fully or partially centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation.
- **Flexibility and Risk in Contract Upgrades**: The project's contracts are upgradable, allowing the administrator to update the contract logic at any time. While this provides flexibility in addressing issues and evolving the project, it also introduces risks if upgrade processes are not properly managed or secured, potentially allowing for unauthorized changes that could compromise the project's integrity and security.

Findings

Vulnerability Details

F-2025-14057 - Recover Function Can Steal User Funds - Medium

Description:

The `recover()` function allows the admin to withdraw any amount of **ATWO** tokens from the contract at any time without restrictions. This creates the same risk as `ATWODistributor`'s emergency withdraw - the malicious admin can steal tokens that are rightfully owed to users based on their presale contributions, rendering the contract insolvent.

Impact:

- Admin can withdraw ATWO tokens that belong to users who purchased in the presale
- Users will be unable to claim their purchased tokens after TGE
- Contract becomes insolvent without warning
- Defeats the purpose of a fair distribution mechanism
- High centralization risk that users should be aware of

Example Scenario:

1. Users purchased 100,000 ATWO total in presale
2. Contract is seeded with 100,000 ATWO for distribution
3. Admin calls `recover(adminAddress, 100000)`
4. TGE happens, users try to claim
5. All `claim()` calls fail due to insufficient balance
6. Users lost their purchased tokens

ATWODistributor.sol:

```
function recover(address to, uint256 amount) external onlyRole(ADMIN_ROLE) {
    atwo.safeTransfer(to, amount); // No checks on user entitlements
}
```

Assets:

- src/contracts/ATWODistributor.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	3/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Remove the recover function entirely, or only allow it after all claims are complete and a sufficient grace period has passed.

Resolution: The finding is fixed in commit `e069e6f` after the suggested fix was implemented in the code. Grace period check was added for the `recover()` function.

F-2025-14062 - Excessive Admin Control Over Critical Staking Parameters - Medium

Description:

The `ATWOStaking` contract grants the admin unrestricted control over multiple critical parameters that directly affect user experience, financial outcomes, and contract functionality. These parameters can be changed at any time without time-locks, user consent, or validation constraints, creating significant trust risks and potential for abuse.

1. Stable Token Can Be Swapped Anytime (`setStableToken`)

```
function setStableToken(address token) external onlyRole(ADMIN_ROLE) {
   if (token == address(0)) revert AddressZero();

   stableToken = IERC20(token); //   Can be changed anytime

   emit StableTokenUpdated(token);
}
```

Issues:

- Admin can swap payment token (e.g., USDC → DAI) at any moment
- Existing `membershipPriceStable` value becomes invalid if decimals differ
- Users expecting to pay in USDC may suddenly need DAI

Impact:

- Price set for USDC (6 decimals) becomes wrong for DAI (18 decimals)
- Users pay wrong amounts (10^12 times difference)
- Inconsistent payment experience
- Potential for exploitation if timed with price changes

2. Membership Price Can Be Changed Without Limits (`setMembershipPriceStable`)

```
function setMembershipPriceStable(uint256 priceStable) external onlyRole(ADMI
N_ROLE) {
   membershipPriceStable = priceStable; //   No boundaries, no limits

   emit MembershipPriceStableUpdated(priceStable);
}
```

Issues:

- No minimum or maximum price constraints
- Can be set to 0 (free memberships)

- Can be set to extremely high values (pricing out users)
- Can be changed mid-sale to favor certain users
- No time-lock or delay

Impact:

- Admin can make memberships free: `setMembershipPriceStable(0)`
- Admin can price out users: `setMembershipPriceStable(1e30)`
- Early buyers pay different prices than late buyers
- No price stability guarantees

3. Team Treasury Can Be Redirected to Steal Fees (`setTeamTreasury`)

```
function setTeamTreasury(bytes32 teamId, address treasury) public onlyRole(ADMIN_ROLE) {
    if (treasury == address(0)) revert AddressZero();

    teamTreasury[teamId] = treasury; //   Can redirect payments anytime

    emit TeamTreasuryUpdated(teamId, treasury);
}
```

Issues:

- Treasury can be changed to admin's personal wallet
- Redirection possible even after users commit to specific team

Impact:

- Admin calls `setTeamTreasury(teamId, adminWallet)`
- All future membership purchases for that team go to admin
- Legitimate team never receives funds

4. NFT Contract Can Be Swapped to DoS Users (`setMemberNft`)

```
function setMemberNft(bytes32 leagueId, bytes32 teamId, address nft) public onlyRole(ADMIN_ROLE) {
    if (nft == address(0)) revert AddressZero();

    memberNft[leagueId][teamId] = nft; //   Can change NFT contract anytime

    emit MemberNftUpdated(leagueId, teamId, nft);
}
```

Issues:

- NFT contract can be swapped after users purchase memberships
- New NFT contract won't recognize old memberships
- Can break `stake()` function which checks `hasMembership()`
- Users who paid for membership suddenly can't stake

5. League Windows Can Be Manipulated to Lock Funds (`setLeagueWindow`)

```
function setLeagueWindow(bytes32 leagueId, uint64 startTime, uint64 endTime)
public onlyRole(ADMIN_ROLE) {
    if (startTime == 0 || endTime == 0 || startTime >= endTime)
        revert InvalidLeagueWindow(leagueId, startTime, endTime);

    leagueWindowById[leagueId] = LeagueWindow({ startTime: startTime, endTime:
endTime }); //   Can extend indefinitely

    emit LeagueWindowUpdated(leagueId, startTime, endTime);
}
```

Issues:

- Admin can extend `endTime` indefinitely: `setLeagueWindow(leagueId, start, type(uint64).max)`
- No maximum duration limit
- No restrictions on how many times windows can be modified
- Users can't unstake during actual league window

Assets:

- src/contracts/ATWOStaking.sol [https://github.com/arenatwo/smart-contracts]

Status: Mitigated

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Implement multiple layers of protection to limit admin power and protect users:

1. Add constraints (upper/lower boundaries or conditional checks) to parameter changes
2. Implement timelock for critical changes
3. Use Multi-Signature wallet for admin

4. Make critical parameters immutable after initialization
5. And finally, document trust model clearly

Resolution: The finding is mitigated with the client's statement below:

> We are using a Multi-Signature for admin (3) already as well as documentation (5).
>
> We don't want to make critical parameters immutable (2) just in case we missinitialize it or we really must change them in the future for safety reasons.
>
> Regarding `setStableToken`, `setMembershipPriceStable`, `setTeamTreasury` and `setMemberNft` functions we acknowledge and accept all concerns regarding the parameter validations, The main point here is that all functions would need to be called multiple times in a short period of time, so we can't add a timelock (2) for them.
>
> We are adding a timelock to limit the upgradability of the Proxy.

F-2025-14066 - Excessive Emergency Withdraw Can Steal User Funds - Medium

Description: The `ATWOVesting.emergencyWithdraw()` function allows the admin to withdraw any amount of tokens from the contract at any time, with no restrictions. This can be used to steal vested tokens that rightfully belong to users, rendering the contract insolvent and preventing users from claiming their entitled tokens.

Impact:

- Admin can withdraw tokens reserved for user vesting schedules
- Users will be unable to claim vested tokens when the contract has insufficient balance
- Contract becomes insolvent without any warning or recourse for users
- While named "emergency," there are no actual emergency conditions checked
- Complete centralization risk that defeats the purpose of a vesting contract

ATWOVesting.sol:

```
function emergencyWithdraw(address to, uint256 amount) external onlyRole(ADMIN_ROLE) {
    if (to == address(0)) revert ZeroAddress();
    if (amount == 0) revert AmountZero();

    vestedToken.safeTransfer(to, amount); // No checks on reserved amounts

    emit EmergencyWithdraw(to, amount);
}
```

Assets:

- src/contracts/ATWOVesting.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: There are different approaches can be taken into account to resolve this vulnerability:

1. Track reserved amounts and only allow withdrawing excess.
2. Add time-lock for emergency withdraws
3. Remove the emergency withdraw function entirely and rely on the contract upgrade mechanism (UUPS) for true emergencies.

Resolution: The finding is fixed in commit `e069e6f` after the suggested fix (removal of `emergencyWithdraw()` function) was applied in the code.

F-2025-14071 - No On-Chain Vote Tallying - Medium

Description:

The `ATWOVoting.vote()` function collects tokens from users and emits events, but **does not maintain any on-chain state tracking of vote counts**.

This means:

1. No on-chain record of how many votes each option received
2. No way to query vote results trustlessly on-chain
3. Results depend entirely on off-chain event indexing
4. No ability to build on-chain logic that depends on voting outcomes
5. If events are missed or misindexed, votes are effectively lost

This is a fundamental design flaw - even in a "pay-per-vote" model where users can vote multiple times, the contract should still track total votes per option to determine which option won.

ATWOVoting.sol:

```
function vote(
    bytes32 leagueId,
    bytes32 matchId,
    bytes32 questionId,
    bytes32 optionId,
    uint256 votes
) public whenNotPaused nonReentrant {
    // Validate question is open and option is enabled
    if (!questionOpen[leagueId][matchId][questionId]) {
        revert QuestionNotOpen(leagueId, matchId, questionId);
    }
    if (votes == 0) revert InvalidVoteCount(votes);

    uint256 price = pricePerVote[leagueId][matchId][questionId];
    if (price == 0) revert PricePerVoteNotSet(leagueId, matchId, questionId);
    if (!optionEnabled[leagueId][matchId][questionId][optionId]) {
        revert OptionNotEnabled(leagueId, matchId, questionId, optionId);
    }

    // Collect payment
    uint256 total = votes * price;
    votingToken.safeTransferFrom(msg.sender, treasury, total);

    // ONLY EMITS EVENT - NO STATE UPDATE FOR VOTE COUNTS!
    emit VoteCast(msg.sender, leagueId, matchId, questionId, optionId, votes,
total);
}
```

Assets:

- src/contracts/ATWOVoting.sol [https://github.com/arenatwo/smart-contracts]

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Consider implementing On-Chain vote counting:

```
mapping(bytes32 => mapping(bytes32 => mapping(bytes32 => mapping(bytes32 => u
int256)))) public voteCount;
mapping(bytes32 => mapping(bytes32 => mapping(bytes32 => uint256))) public to
talVotesPerQuestion;

// Update vote function:
function vote(...) external {
    // ... existing logic ...

    uint256 total = votes * price;
    votingToken.safeTransferFrom(msg.sender, treasury, total);

    // ADD THIS:
    voteCount[leagueId][matchId][questionId][optionId] += votes;
    totalVotesPerQuestion[leagueId][matchId][questionId] += votes;

    emit VoteCast(msg.sender, leagueId, matchId, questionId, optionId, votes,
total);
}

// Add view functions:
function getVoteCount(...) external view returns (uint256);
function getTotalVotes(...) external view returns (uint256);
function getWinningOption(...) external view returns (bytes32, uint256);
```

Resolution: The finding was acknowledged by the client with the following notice:

> This was decided by design due to the fact that we collect votes onchain and offchain so it was decided that the main source of trust is going to be the offchain ones.

F-2025-14074 - Admin Can Arbitrarily Decrease User's Vesting Amount Bypassing User Entitlement - Medium

Description:

The `ATWOVesting` contract allows the admin to create linear vesting schedules via the `setSchedule()` function. This function can be called to update the total vesting amount for an existing schedule at any time.

However, there is no restriction preventing the admin from reducing the total vesting amount of an ongoing schedule below the amount already accrued but not yet claimed by the beneficiary. This effectively allows the admin to revoke a user's legitimate entitlement, resulting in loss of tokens that the user has already vested by time progression.

```
function setSchedule(
    address user,
    uint64 startTime,
    uint64 cliffDuration,
    uint64 vestingDuration,
    uint256 total
) external onlyRole(ADMIN_ROLE) {
    if (user == address(0)) revert ZeroAddress();
    // Requirements: startTime > 0, vestingDuration > 0; cliffDuration can be
0
    if (!(startTime > 0 && vestingDuration > 0)) revert InvalidSchedule(start
Time, cliffDuration, vestingDuration);

    VestingSchedule storage schedule = scheduleByUser[user];

    // Preserve claimed value if schedule already exists
    uint256 alreadyClaimed = schedule.claimed;
    // New total must be >= already claimed
    if (total < alreadyClaimed) {
        // Treat as invalid schedule if trying to shrink below claimed
        revert InvalidSchedule(startTime, cliffDuration, vestingDuration);
    }

    schedule.startTime = startTime;
    schedule.cliffDuration = cliffDuration;
    schedule.vestingDuration = vestingDuration;
    schedule.total = total;
    schedule.claimed = alreadyClaimed;

    emit ScheduleUpdated(user, startTime, cliffDuration, vestingDuration, tot
```

```
al);
}
```

The absence of proper validation enables the admin to unilaterally reduce vested amounts, which can lead to:

- Loss of user entitlement to tokens that should already be claimable.
- Inconsistency between the vesting state and the actual user's accrued balance.
- Potential misuse or abuse of administrative privileges, undermining trust and the contract's integrity.

Assets:

- src/contracts/ATWOVesting.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Implement a validation check in `setSchedule()` to ensure the new total vesting amount cannot fall below the user's already vested (accrued) amount. This ensures administrative adjustments cannot invalidate accrued entitlements, preserving fairness and preventing unauthorized reduction of user balances.

Resolution: The finding is fixed in commit `e069e6f` after the suggested fix was implemented in the code. New check now ensure the new total cannot fall below already vested amount.

F-2025-14058 - Initialization and Address Validation Issues - Low

Description:

Multiple contracts in the `ArenaTwo` protocol have initialization issues where critical addresses are either not validated or not initialized at all. These issues range from complete contract failure to deployment-time configuration risks. This consolidated report covers all initialization and address validation findings.

Finding 1: ATWOVoting - Uninitialized Treasury Causes Vote DoS

The `treasury` address is never initialized in the `initialize()` function. in `ATWOVoting` contract. When users call `vote()`, the function attempts to transfer tokens to the treasury address which will be `address(0)`, causing all vote transactions to fail.

Impact:

- All `vote()` calls will fail because treasury is **address(0)**
- Contract is non-functional until admin calls `setTreasury()`
- Users attempting to vote will waste gas on failed transactions
- Contract effectively broken on deployment until fixed

Finding 2: ATWODistributor - Missing Admin Address Validation

The `initialize()` function validates that `_atwo` and `_presale` are not zero addresses, but there is no validation for the `admin` parameter. If a zero address is accidentally passed, the contract would be initialized with no admin, making it impossible to execute admin functions.

Impact:

- If admin is accidentally set to **address(0)**, contract becomes partially locked
- Admin-only functions (`pause`, `unpause`, `setTgeTimestamp`, `seed`, `recover`) become inaccessible
- Would require contract redeployment to fix

Assets:

- src/contracts/ATWOVoting.sol [https://github.com/arenatwo/smart-contracts]
- src/contracts/ATWODistributor.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: As a recommendation, consider implementing all checks for the affected functions to avoid any potential problems.

Resolution: The finding is partially fixed in commit `e069e6f`, as client mentioned that:

> Finding 1: The treasury wallet will be set up right after the deployment, this is acknowledge by the team and accepted
>
> Finding 2: Fixed with the proposed recommendation.

F-2025-14065 - Use of Unsafe transferFrom Instead of safeTransferFrom - Low

Description:

The `ATWOStaking.purchaseMembership()` function uses raw `transferFrom()` instead of OpenZeppelin's `safeTransferFrom()` wrapper. This is inconsistent with the rest of the contract, which correctly uses **SafeERC20** for all other token operations.

While the admin controls which token is set as `stableToken`, many widely-used production tokens (like **USDT** on some chains) don't strictly follow the **ERC20** standard. If the admin configures such a token, the payment transfer can silently fail without reverting. The `transferFrom` call returns `false`, but since the return value is not checked, execution continues and the membership NFT is minted anyway.

The contract already imports and uses **SafeERC20** everywhere else:

- Line 235: `stakingToken.safeTransferFrom(user, address(this), amount);`
- Line 300: `stakingToken.safeTransfer(user, unstakeAmount);`
- Line 208: `stableToken.transferFrom(user, treasury, price);` ← **Only inconsistent usage**

ATWOStaking.sol:

```
function purchaseMembership(bytes32 leagueId, bytes32 teamId) public whenNotP
aused nonReentrant {
    address user = msg.sender;

    // Ensure membership NFT contract is configured for league/team
    address memNft = memberNft[leagueId][teamId];
    if (memNft == address(0)) revert MemberNftNotSet(leagueId, teamId);

    // Block purchase if the user already owns a membership
    bool isMember = IATWOMemberNFT(memNft).hasMembership(user);
    if (isMember) revert AlreadyMember(user, leagueId, teamId);

    // Stable token must be configured and global price must be set
    if (address(stableToken) == address(0)) revert StableTokenNotSet();

    uint256 price = membershipPriceStable;
    if (price == 0) revert MembershipPriceNotSet();

    // Treasury must be configured for the team
    address treasury = teamTreasury[teamId];
    if (treasury == address(0)) revert TreasuryNotSet(teamId);
```

```
    // Uses unsafe transferFrom without checking return value
    stableToken.transferFrom(user, treasury, price);

    // Mint membership NFT even if transfer failed!
    IATWOMemberNFT(memNft).mintMembership(user);
}
```

Assets:

- src/contracts/ATWOStaking.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Change the unsafe `transferFrom()` function with secure `safeTransferFrom()` for consistency with the rest of the contract.

Resolution: The finding is fixed in commit `e069e6f` after the suggested fix was implemented in the code. Unsafe `transferFrom()` method was replaced with safer `safeTransferFrom()` method.

F-2025-14073 - Missing Reserve Check Allows Creation of Underfunded Vesting Schedules - Low

Description:

The `ATWOVesting` contract enables the contract admin to create linear vesting schedules using the `setSchedule()` function. However, the current implementation does not verify whether the contract holds a sufficient token balance to cover the total vesting amount for the new schedule.

As a result, the admin can create vesting schedules whose cumulative obligations exceed the contract's available token balance. This can lead to insufficient reserves when beneficiaries attempt to claim their vested tokens in the future, causing claim failures or partial payouts.

```
function setSchedule(
    address user,
    uint64 startTime,
    uint64 cliffDuration,
    uint64 vestingDuration,
    uint256 total
) external onlyRole(ADMIN_ROLE) {
    if (user == address(0)) revert ZeroAddress();
    // Requirements: startTime > 0, vestingDuration > 0; cliffDuration can be
0
    if (!(startTime > 0 && vestingDuration > 0)) revert InvalidSchedule(start
Time, cliffDuration, vestingDuration);

    VestingSchedule storage schedule = scheduleByUser[user];

    // Preserve claimed value if schedule already exists
    uint256 alreadyClaimed = schedule.claimed;
    // New total must be >= already claimed
    if (total < alreadyClaimed) {
        // Treat as invalid schedule if trying to shrink below claimed
        revert InvalidSchedule(startTime, cliffDuration, vestingDuration);
    }

    schedule.startTime = startTime;
    schedule.cliffDuration = cliffDuration;
    schedule.vestingDuration = vestingDuration;
    schedule.total = total; // NO CHECK if the contract has enought reverves
to cover vesting
    schedule.claimed = alreadyClaimed;

    emit ScheduleUpdated(user, startTime, cliffDuration, vestingDuration, tot
```

```
al);
}
```

Lack of a reserve validation check introduces the risk of over-allocating vesting commitments beyond the contract's token holdings, resulting in:

- Inability to fulfill legitimate vesting claims.
- Potential disputes between users and administrators.
- Loss of confidence in the contract's reliability and accounting integrity.

Assets:

- src/contracts/ATWOVoting.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Implement a validation step in `setSchedule()` to ensure the contract balance can cover all vesting liabilities, including both existing entitlements and the new schedule amount.

This ensures that all created vesting schedules remain fully backed by actual token reserves, maintaining consistency between on-chain obligations and available balances.

Resolution: The finding is fixed in commit `ca89ed1` after the suggested fix was implemented in the code. The new logic ensures that the contract maintains sufficient balance and properly accounts for released amounts when updating the total vesting obligations.

F-2025-14067 - Floating Pragma - Info

Description: In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.xx`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.22`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.xx`, explicitly define the version with pragma solidity `0.8.22;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: Solidity GitHub

releases or Solidity Bugs by Version. Choose a compiler version with a good track record for stability and security.

Resolution: The finding is fixed in commit `e069e6f` after the pragma version was fixed to `0.8.22`.

F-2025-14070 - Use of `public` Visibility Instead of `external` for External-Only Functions - Info

Description:

Multiple functions across the codebase use `public` visibility when they are only ever called externally. Using `external` visibility instead of `public` would result in gas savings, as `external` functions can read directly from calldata, while `public` functions must copy **calldata** to memory.

Functions marked as `public` can be called both internally (from within the contract) and externally (from transactions or other contracts). However, if a function is never called internally, using `external` is more gas-efficient.

Gas savings per function call: ~200-500 gas depending on:

- Number and type of parameters
- Size of arrays/structs passed
- Overall calldata complexity

Affected Contracts:

- ATWOVoting
- ATWOStaking

Assets:

- src/contracts/ATWOVoting.sol [https://github.com/arenatwo/smart-contracts]
- src/contracts/ATWOStaking.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Change all external-only functions to `external` to consume less gas.

Resolution: The finding is fixed in commit `e069e6f` after the most of `public` functions were replaced with `external` function visibility.

F-2025-14072 - Unused Custom Error and Import - Info

Description:

There is unused custom error and library import within the codebase, including:

Custom errors:

- `IATWOOGPassSale.AmountZero()`

Unused import in `ATWOMemberNFT`:

```
import {Strings} from "@openzeppelin/contracts/utils/Strings.sol";
```

These custom error and library import are defined but never thrown or referenced anywhere in the current implementation of the contracts. Unused code contributes to unnecessary bytecode size, potentially increasing deployment and execution costs. It can also reduce code clarity and maintainability over time. While the impact may be minor, it contributes to reduced optimization and could be misleading to users.

Assets:

- src/contracts/ATWOMemberNFT.sol [https://github.com/arenatwo/smart-contracts]
- src/interfaces/IATWOOGPassSale.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Consider removing unused custom error and library import to keep the codebase clean and optimized. Alternatively, if these errors are intended for use, ensure they are implemented in appropriate locations where they add meaningful validation.

Resolution: The finding is fixed in commit `e069e6f` after the unused import was cleared out for the `ATWOMemberNFT` contract.

F-2025-14076 - Lack of Event Emitting for Key State Changes - Info

Description:

Several functions within the protocol perform meaningful state changes and token movements without emitting events, which hinders off-chain observability:

- `recover()` in `ATWODistributor` and `ATWOPublicSale` rescues ERC-20 balances.
- `seed()` in `ATWODistributor` and `ATWOPublicSale` funds contracts for TGE/public sale flows.
- `purchaseMembership()` in `ATWOStaking` records a membership NFT purchase.

The absence of an event makes it difficult to track admin-specific actions off-chain, which reduces transparency and complicates monitoring and auditing. It is recommended to emit events for this operation to ensure proper visibility, support off-chain indexing, and facilitate debugging, especially for critical administrative and user-facing fund transfers.

Assets:

- src/contracts/ATWODistributor.sol [https://github.com/arenatwo/smart-contracts]
- src/contracts/ATWOPublicSale.sol [https://github.com/arenatwo/smart-contracts]
- src/contracts/ATWOStaking.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider emitting events within the mentioned affected functions to enhance transparency and ensure reliable off-chain tracking of key state changes. Emitting events not only improves monitoring of critical storage

variable updates and key operation execution but also supports indexing services, facilitates auditing, and provides better visibility into administrative actions.

Resolution: The finding is fixed in commit `e069e6f`. Events for crucial functionalities were implemented as suggested.

F-2025-14077 - Redundant Token Address Comparison in recover() Function - Info

Description:

The `recover()` function of the `ATWOPublicSale` contract allows rescuing ERC20 tokens, including the ATWO token itself. The function currently performs a redundant comparison to check whether the provided token address equals the stored atwo address. If it matches, the transfer is executed via the atwo variable; otherwise, a new IERC20 instance is created from the provided address.

Since both branches call the same `safeTransfer()` function under the same ERC20 interface, this conditional check introduces unnecessary gas overhead and an additional storage read for atwo, without providing any functional benefit.

```
function recover(address token, address to, uint256 amount) external onlyRole
(ADMIN_ROLE) {
    if (token == address(atwo)) {
        atwo.safeTransfer(to, amount);
    } else {
        IERC20(token).safeTransfer(to, amount);
    }
}
```

This redundant comparison and storage access slightly increase gas costs and code complexity without improving functionality. It also reduces readability and maintainability by introducing an unnecessary branching condition.

Assets:

- src/contracts/ATWOPublicSale.sol [https://github.com/arenatwo/smart-contracts]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Simplify the logic by removing the redundant if-else check and directly invoking the `safeTransfer()` call using the provided `token` address. This eliminates the extra storage read and streamlines execution:

```
IERC20(token).safeTransfer(to, amount);
```

Resolution: The finding is fixed in commit `e069e6f`. The transfer logic was simplified.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/arenatwo/smart-contracts/tree/hacken-2025-10-31
Initial Commit	3d8f05bdb5a9b74440ac1fba9b719dec44d7c459
Remediation Commit	ca89ed19f5cfdbb261aa66f1becb394a86d21bb3
Whitepaper	N/A
Requirements	README.md
Technical Requirements	README.md

Asset	Type
src/contracts/ATWO.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/contracts/ATWODistributor.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/contracts/ATWOMemberNFT.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/contracts/ATWOPresale.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/contracts/ATWOPublicSale.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/contracts/ATWOStaking.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/contracts/ATWOVesting.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/contracts/ATWOVoting.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/interfaces/IATWODistributor.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/interfaces/IATWOMemberNFT.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/interfaces/IATWOPresale.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/interfaces/IATWOPublicSale.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/interfaces/IATWOStaking.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/interfaces/IATWOVesting.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract

Asset	Type
src/interfaces/IATWOVoting.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract
src/interfaces/IChainlinkVRFWrapper.sol [https://github.com/arenatwo/smart-contracts]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.